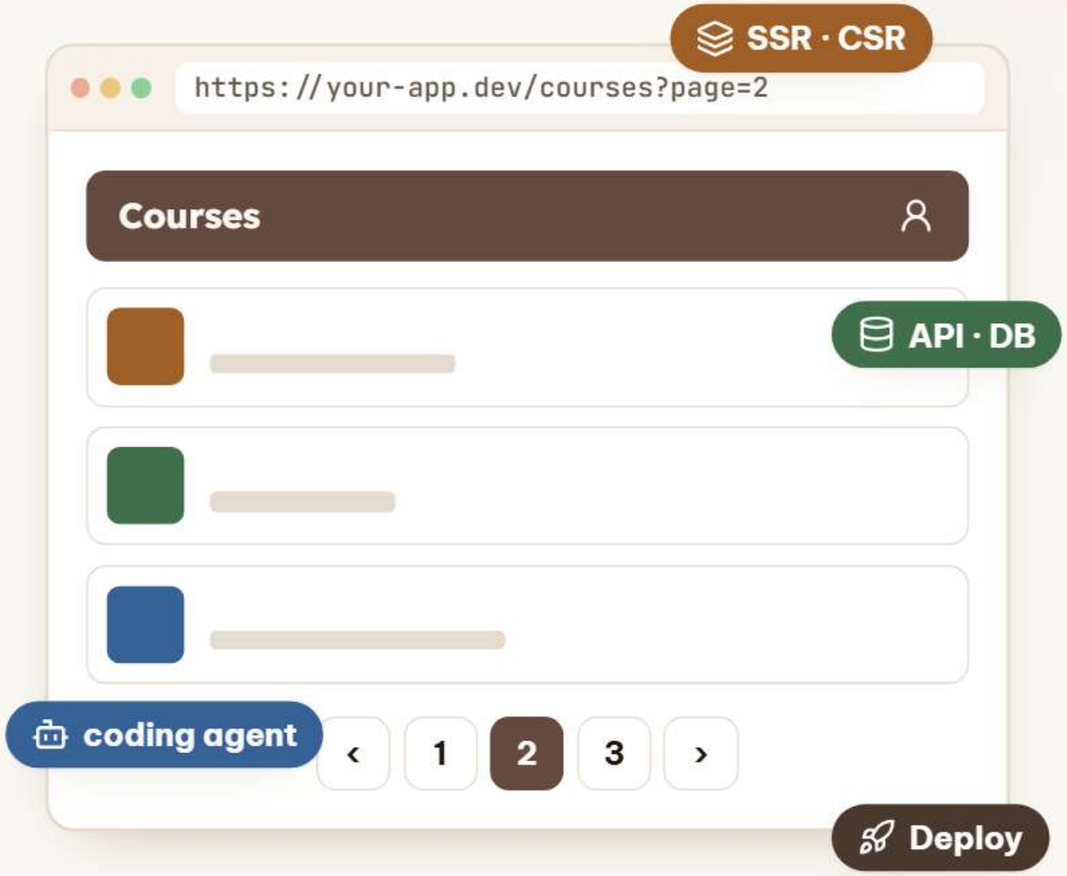
Course Syllabus · 2026–2027 CSC13008

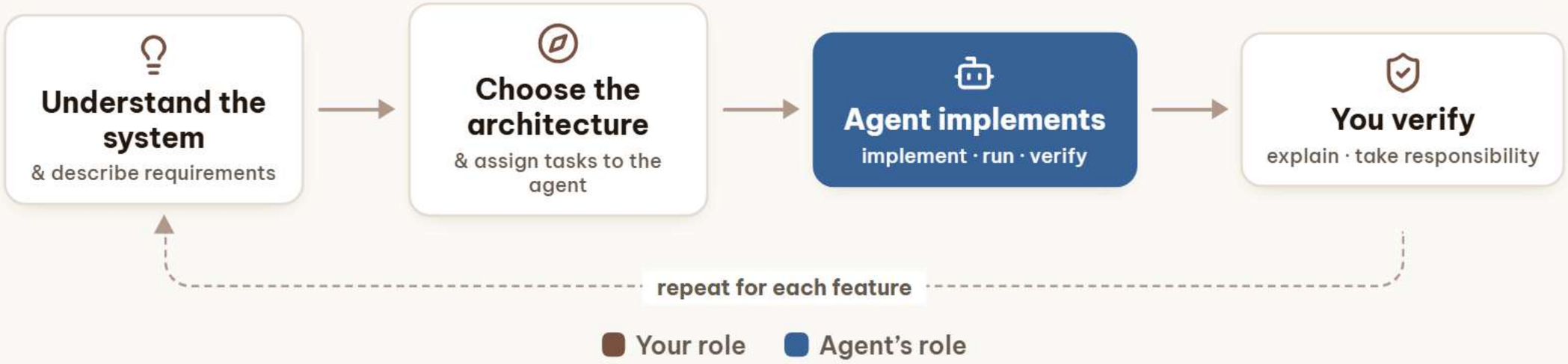
Web Application Development

Understand the web · Build real products
Work with a **coding agent**

11	22	100′	
weeks	blocks	per block	teamwork



Agents write code fast. You own the product.

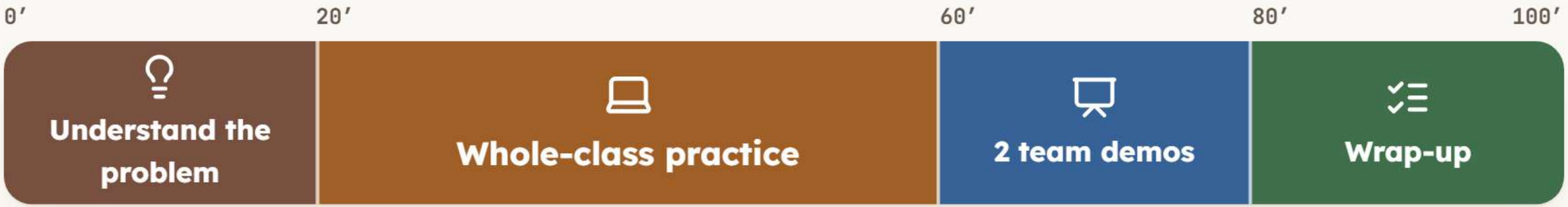


An agent is an **implementation tool** — not a source of truth.

Don't race the agent at typing code — use that time to **explore more technologies**.

LEARNING RHYTHM · 2 BLOCKS PER WEEK

What happens in your 100-minute block?



- One big question
- Mental model
- Short demo

- Same task and starter project
- Use coding agents freely
- The result **must run**

- Teams chosen **at random**
- 10 minutes per team

- Compare two approaches
- Correct misconceptions
- 3–5 key takeaways

One task — each team, its own solution



YOUR TEAM'S DEMO · 10'

- Show the result
- Explain your approach
- Problems encountered
- How the agent helped
- Notable technical choices

Any team may be called — always be ready to demo!

ROADMAP

11 weeks · 6 phases · from static pages to production

PHASE 1 · W1
Web Platform

- HTML/CSS/JS
- Browser ↔ Server
- Static hosting

PHASE 2 · W2–4
Rendering & UI

- SSR
- CSR
- Component
- Styling system
- SSG
- Hydration
- Hybrid

PHASE 3 · W5–7
App Structure & Data

- Routing
- Query param
- Pagination
- Data fetching
- API
- Search · Sort · Filter
- Database
- Auth
- Full-stack flow

PHASE 4 · W8
Agentic Dev

- Requirement → Spec
- Agent implement
- Browser / E2E verify
- **From W8: the default workflow**

PHASE 5 · W9–10
Practical Web Capabilities

- Upload
- Object Storage
- CDN
- Realtime
- WYSIWYG
- Third-party

PHASE 6 · W11
Ship!

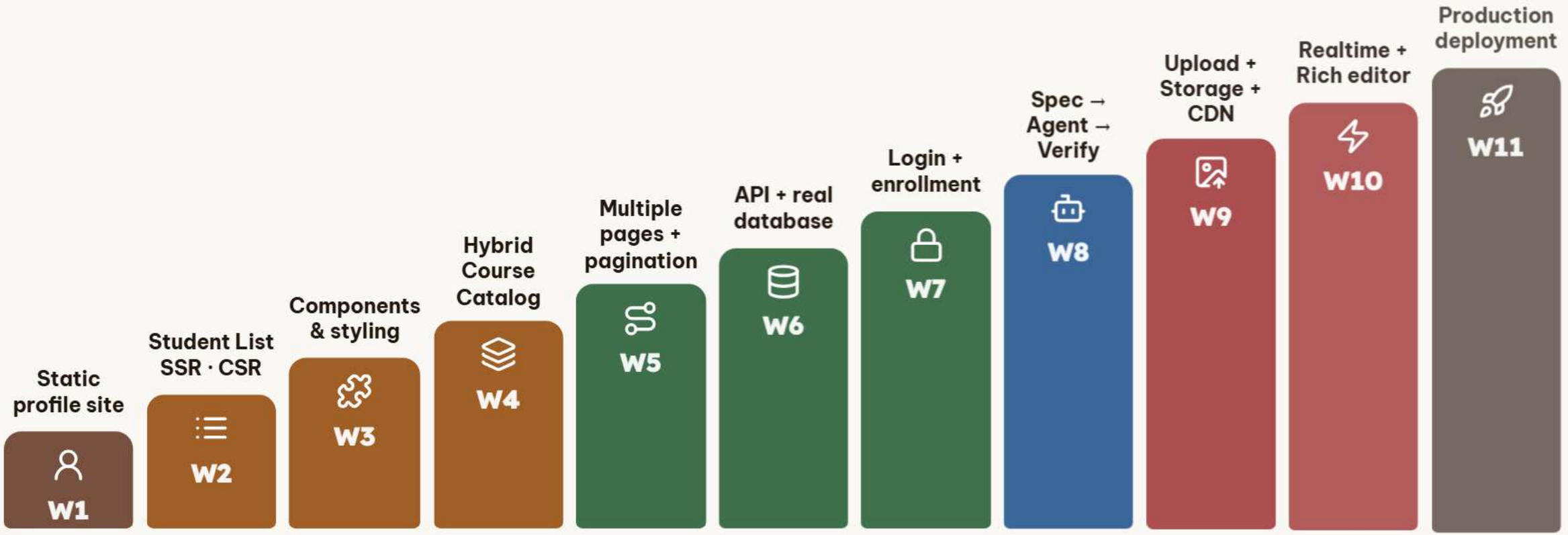
- Build
- Runtime
- Deployment

1 2 3 4 5 6 7 8 9 10 11

YOUR PRODUCT

Each week, your app gains another layer

Shared data model: Student · Course · Enrollment



Agents write most of the code — what must you understand?

THE BROWSER RECEIVES THREE THINGS

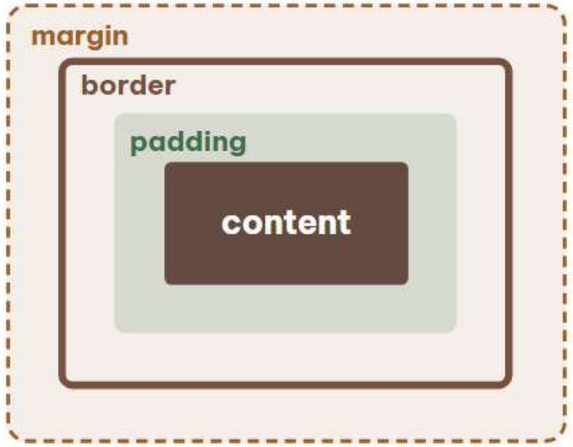
HTML
Structure · content

CSS
Presentation · layout · responsive design

JavaScript
Behavior · interaction

KEYWORDS: selector, class, box model, layout, responsive, Bootstrap

BOX MODEL



SHARED EXERCISE · 40'

Use an agent to build a static profile site

- HTML + CSS
- Responsive
- 1 JS interaction
- 2–3 sections
- Plain CSS / CSS framework

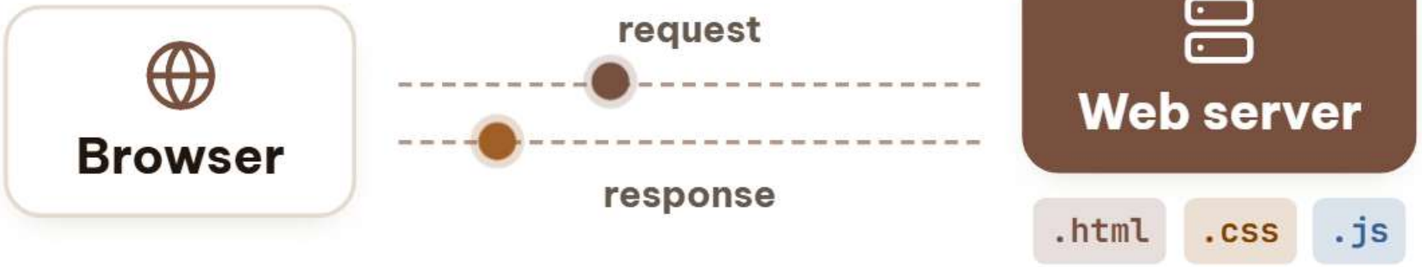
DELIVERABLE
Static site running locally

KEY TAKEAWAYS

Agents generate code fast — you must still understand the output.

An agent is a tool, not a source of truth.

Enter a URL — what happens between the browser and the server?



WHERE DOES HTML COME FROM?

Static
Prebuilt HTML

SSR
Server generates HTML

CSR
Browser runs JS to build the UI

SHARED EXERCISE · 40'

Host your Block 1 site with Nginx / Caddy

- No dev server
- Serve HTML/CSS/JS correctly
- Inspect DevTools › Network

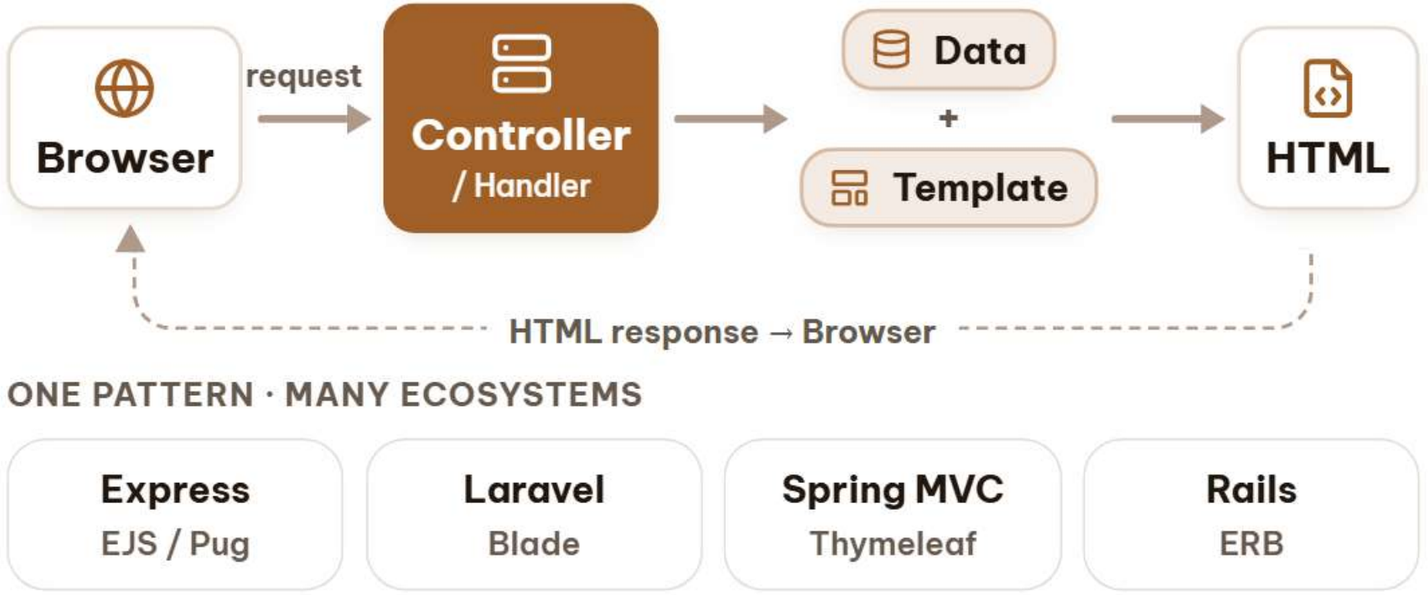
DELIVERABLE

Site served by Nginx/Caddy · config · Network screenshot

HOMEWORK · WEEK 1

Finish the static site: responsive · 1 JS interaction · Nginx/Caddy hosting · brief run instructions

HTML is not prebuilt — how does the server generate it from data?



ONE PATTERN · MANY ECOSYSTEMS

Express	Laravel	Spring MVC	Rails
EJS / Pug	Blade	Thymeleaf	ERB

SHARED EXERCISE · 40'

Server-rendered Student List

- `/students` Student list
- Add Student form
- Submit → server processes
- Re-render HTML

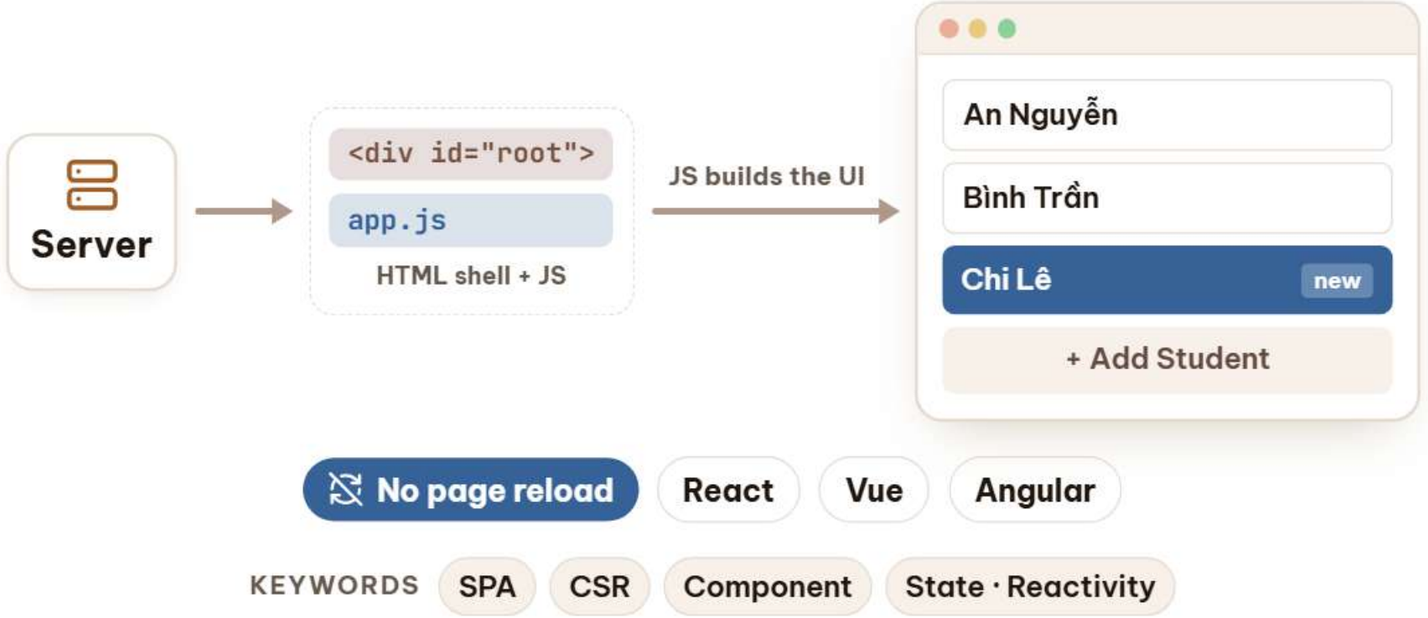
DELIVERABLE

Working SSR Student List

KEY TAKEAWAYS

Don't memorize every framework's syntax — recognize the shared pattern.

What if the browser runs JavaScript to render the UI?



No page reload | React | Vue | Angular

KEYWORDS SPA | CSR | Component | State · Reactivity

SHARED EXERCISE · 40′

The same Student List — CSR version

- List + add Student
- No page reload
- Suggested: React
- Bonus: Vue / Angular version

DELIVERABLE

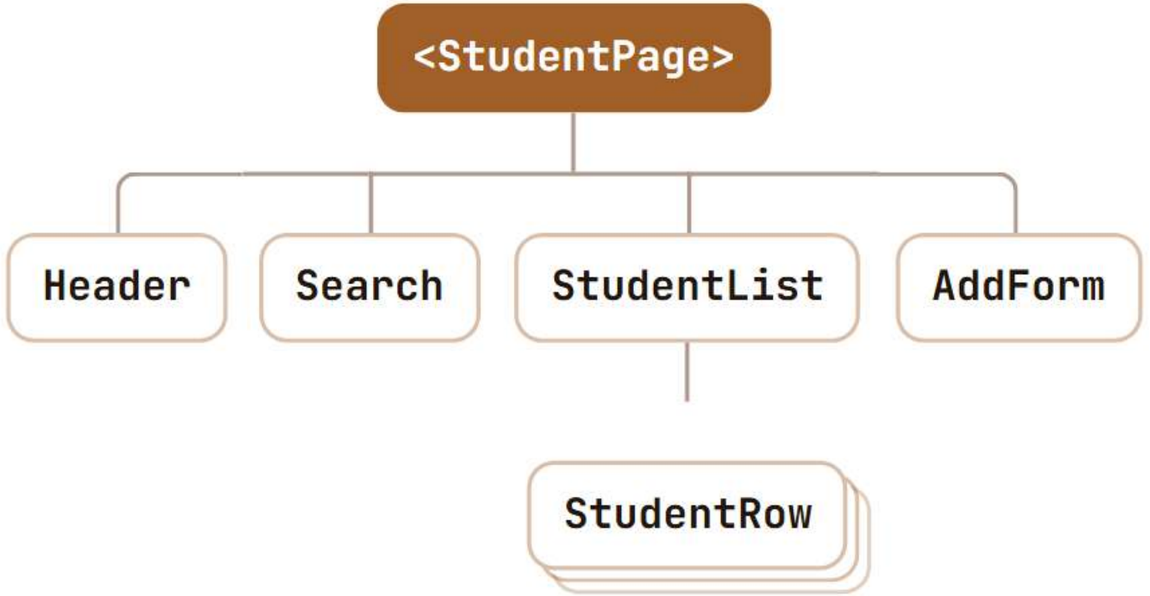
Working CSR Student List

KEY TAKEAWAYS

~~Laravel = SSR, React = CSR~~

SSR / CSR are rendering models — one framework can support multiple models.

As the UI grows, how do you break it down & reuse it?



Props
data flows down

Event
events flow up

State
data changes

SHARED MINDSET React Vue Angular Composition

SHARED EXERCISE · 40′

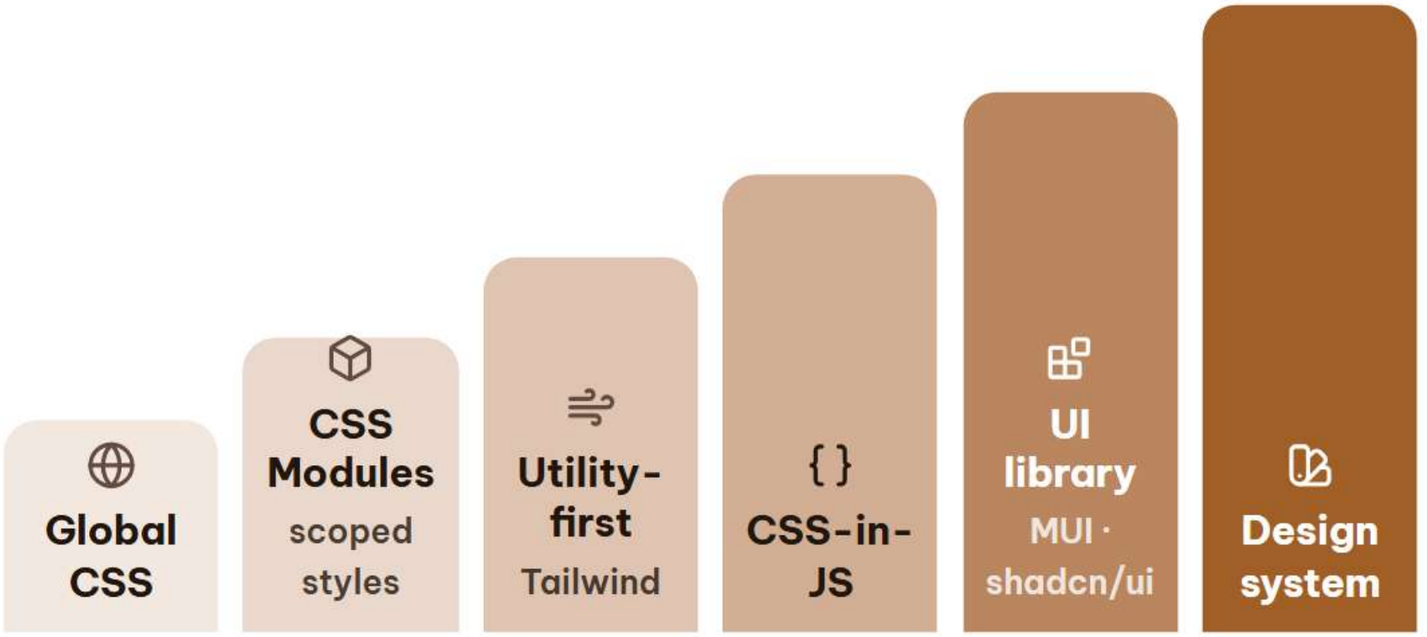
Component-based Student Management page

- Header
- Search
- Student List
- Row / Card
- Add form
- ≥ 1 state interaction

DELIVERABLE
Working component-based UI

FOCUS ON
The component mental model — not syntax details.

How do you organize styles to scale across components?



SHARED EXERCISE · 40′

Two styling versions of Student Management

A	B
Tailwind	Component library

Same requirements · Same data · Responsive

DELIVERABLE

Two working UI versions

KEY TAKEAWAYS

W1: how do you style one page?

W3: how do you organize styles for the whole app?

Where is HTML generated — and when?

	At build time	Server · per request	Browser
CSR	–	–	JS builds the UI
SSR	–	Generate HTML	display
SSG	Prebuild HTML	serve statically	display
SSR/SSG + Hydration	HTML comes first		JS enables interaction

Hybrid each page can use a different approach

SHARED EXERCISE · 40′

Course Catalog × 2 rendering approaches

CSR version | SSR or SSG version

Same data · same UI

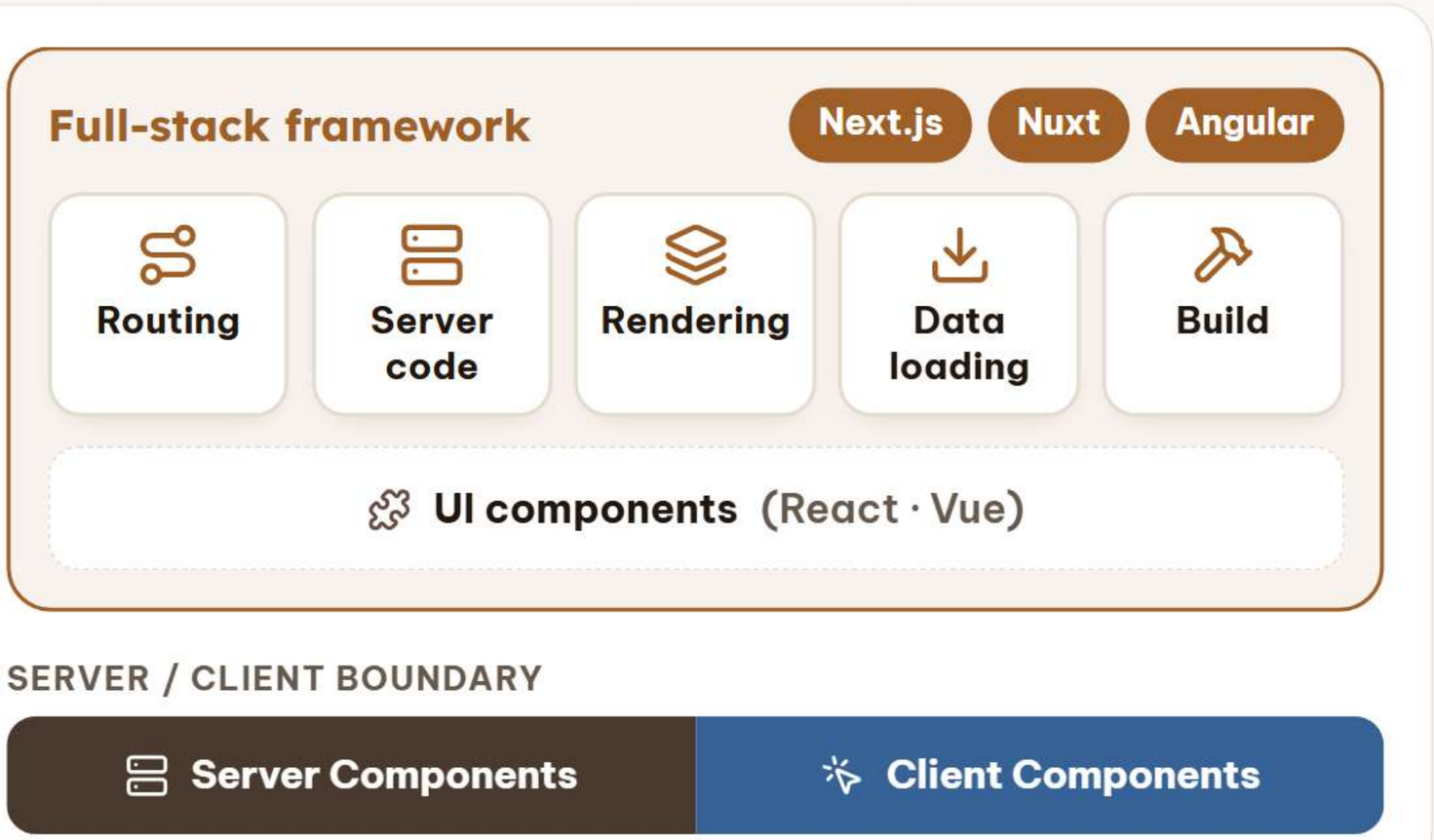
INSPECT THE DIFFERENCES

Reload | View Source | Network | Disable JS

DELIVERABLE

Same feature · 2 rendering models

Is React / Vue enough? What does a Full-stack framework add?



SHARED EXERCISE · 40'

Hybrid mini app with a full-stack framework

- 1 server-rendered page
- 1 interactive client component
- ≥ 2 pages with navigation
- Load data on the server or client

DELIVERABLE

Working hybrid mini app

When the URL changes, which page & state should the app display?



SHARED EXERCISE · 40'

Course app → multiple pages + query state

`/courses` `/courses/:id`

`/students` `/students/:id`

`?page ?search ?sort`

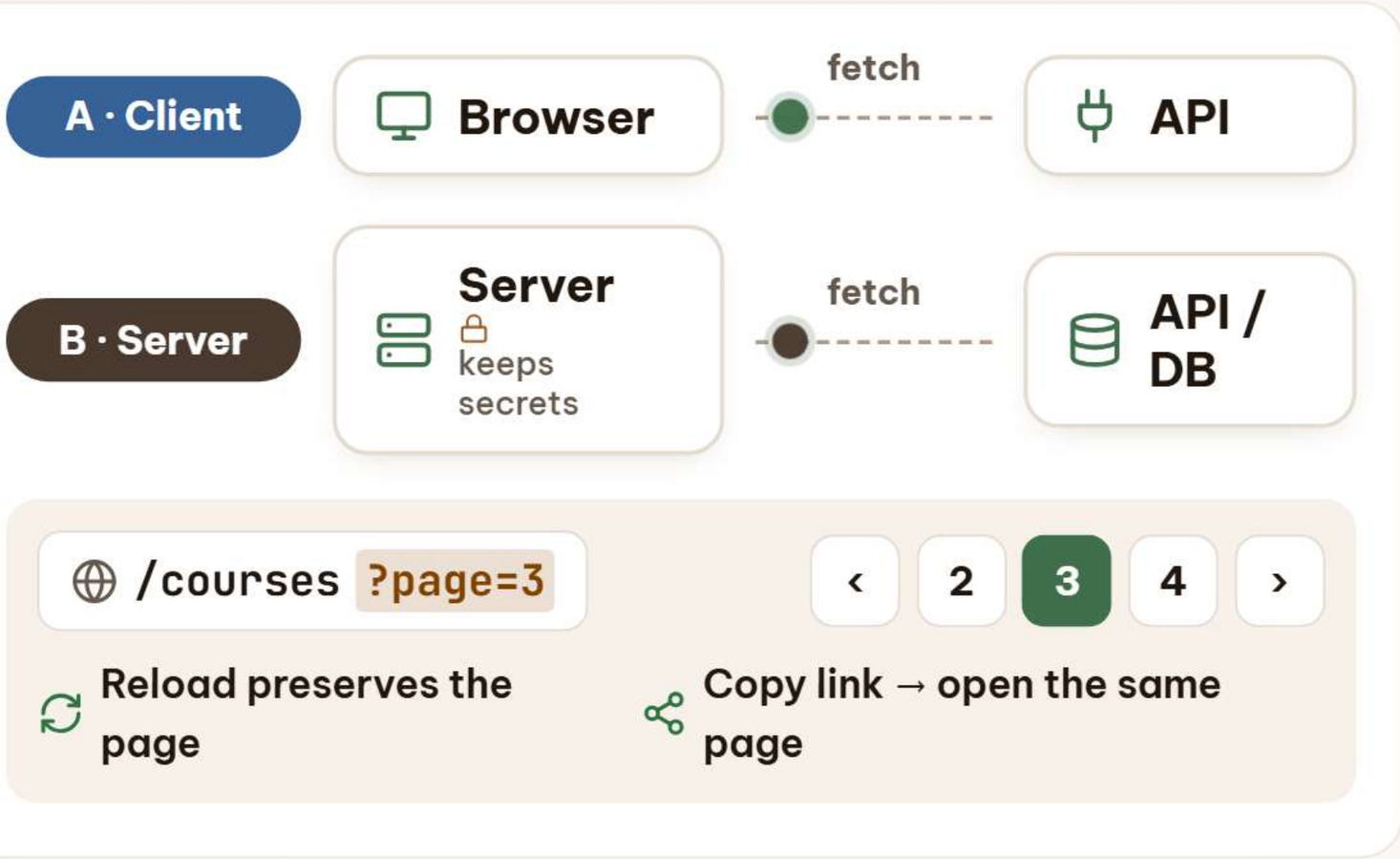
DELIVERABLE

Multi-route app · UI state in the URL

KEY TAKEAWAYS

The URL selects a page and stores state: page · search · sort · filter.

Fetch data in the browser or on the server? Where is the current page stored?



SHARED EXERCISE · 40'

Paginated Course List — 2 approaches

- A fetch on the client
- B server reads ?page
- Page in the URL
- Page survives reload
- Shareable link

DELIVERABLE

Two paginated Course List versions

KEYWORDS loading cache secret server/client boundary

Tens of thousands of Students — should the API return them all at once?

```
GET /students?page=2&pageSize=20
GET /students?search=an
GET /students?sort=name
```

→

```
{
  "items": [ …20 ],
  "page": 2,
  "pageSize": 20,
  "total": 537
}
```

537 Students → 27 pages

‹ 1 2 3 … 27 ›

GET POST PUT/PATCH DELETE | page/pageSize limit/offset cursor

KEYWORDS REST resource endpoint status code validation API contract

SHARED EXERCISE · 40'

Student API with pagination · search · sort

List · Detail | Create · Update · Delete | page / pageSize | Search by name | Sort by ≥ 1 field | Metadata for the UI

DELIVERABLE

Working API · tested with an HTTP client

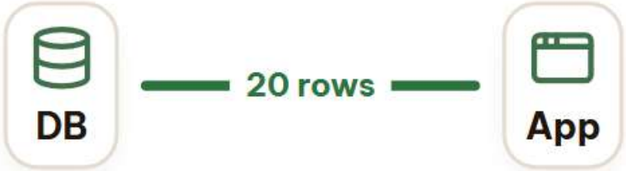
Let the database return one page — don't load everything and slice

Load everything, then `.slice()`



Slow · wastes RAM · wastes bandwidth

Paginate at the DB



`LIMIT 20 OFFSET 20 · COUNT(*)`

`Student` 1 — n `Enrollment` n — 1 `Course`

KEYWORDS: schema · ORM · migration · seed · constraint · index

SHARED EXERCISE · 40'

Replace in-memory storage with a real DB

- Seed hundreds to thousands of rows
- Query exactly 1 page
- Total count
- Search · Sort
- Relationship

DELIVERABLE

Paginated API backed by a real DB

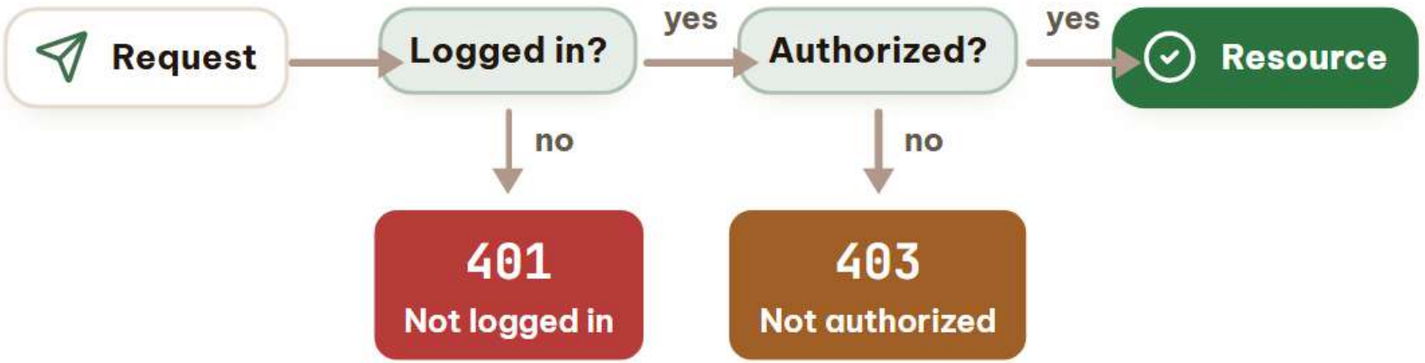
PAGINATION ACROSS EVERY LAYER

URL → UI state → API params → DB query → metadata → UI

How does the app know who you are — and what you may do?

Authentication
Who are you? · session / cookie

Authorization
What may you do? · role



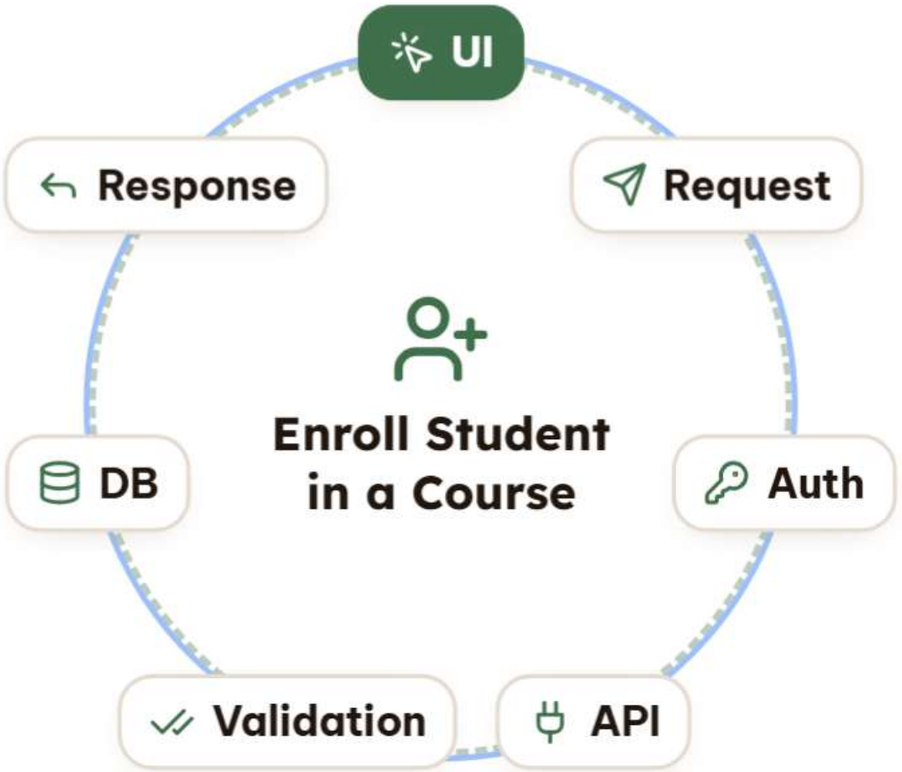
SHARED EXERCISE · 40'

Add login & roles to the app

- Lecturer
- Admin
- Login / logout
- Protected page
- Protected API
- ≥ 1 Admin-only action

DELIVERABLE
Working auth flow

How does one click travel through UI, auth, API, DB?



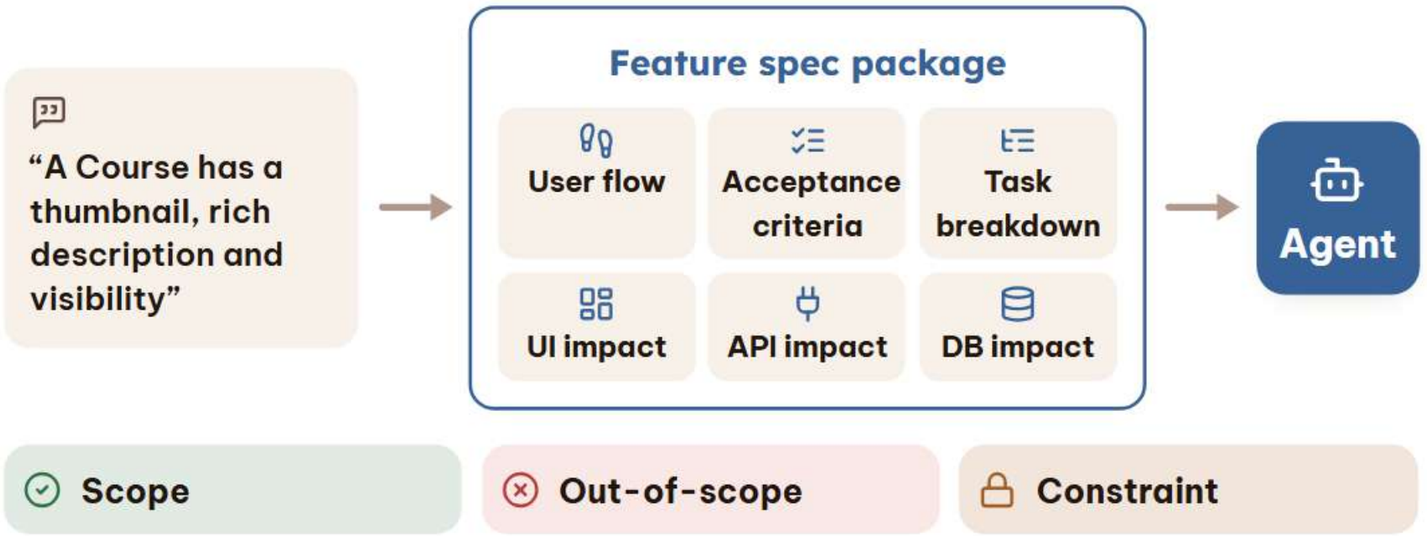
SHARED EXERCISE · 40'

Feature: enroll a Student in a Course

1. Lecturer selects a Student & Course
2. Submit → auth check → validation
3. Save Enrollment
4. Update the UI

DELIVERABLE
One complete full-stack feature

How do you describe requirements so the agent gets it right?



SHARED EXERCISE · 40'

Write a feature spec package as a team

- Based on the task requirements
- All 6 sections
- Break into agent-ready tasks

DELIVERABLE
A spec ready to hand to the agent

Delegate a feature to the agent — what do you control?

Monitor the whole process — intervene when the agent goes off track



THE AGENT MAY

- read the repo
- edit multiple files
- run commands
- run the app
- browser / E2E
- fix its own errors

SHARED EXERCISE · 40'

Have the agent implement your Block 1 spec

- Agent works end-to-end
- Team intervenes if it goes off track

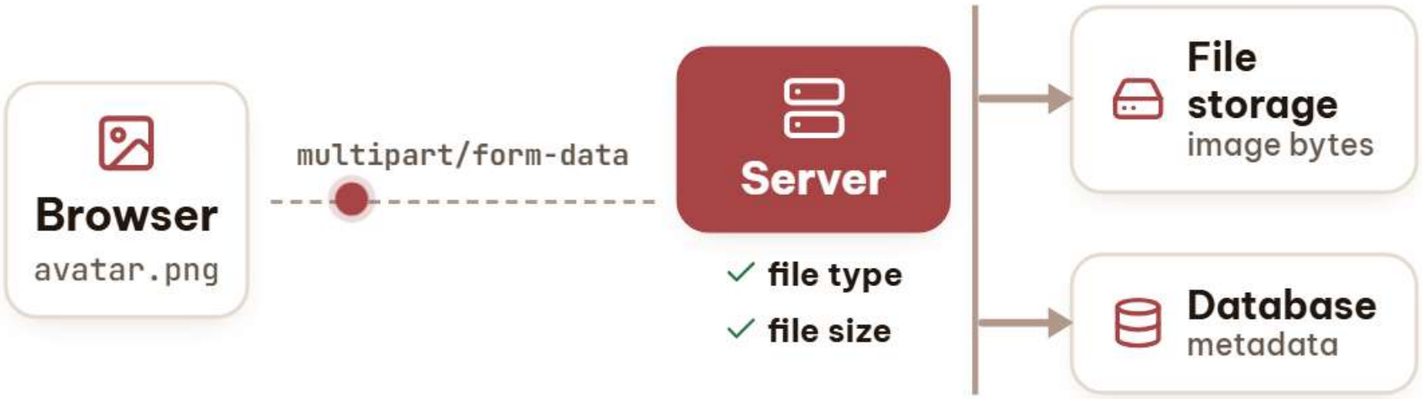
DELIVERABLE

Working feature · user flow verified

AGENTIC CODING IN THIS COURSE

Agents must not just write code — they must run & verify the feature.

Upload a file — where does it go, and what does the app store?



KEY	TYPE	SIZE	OWNER
avatars/42.png	image/png	120 KB	student #42

UNLIKE JSON — send bytes, not just text — validate before saving

SHARED EXERCISE · 40'

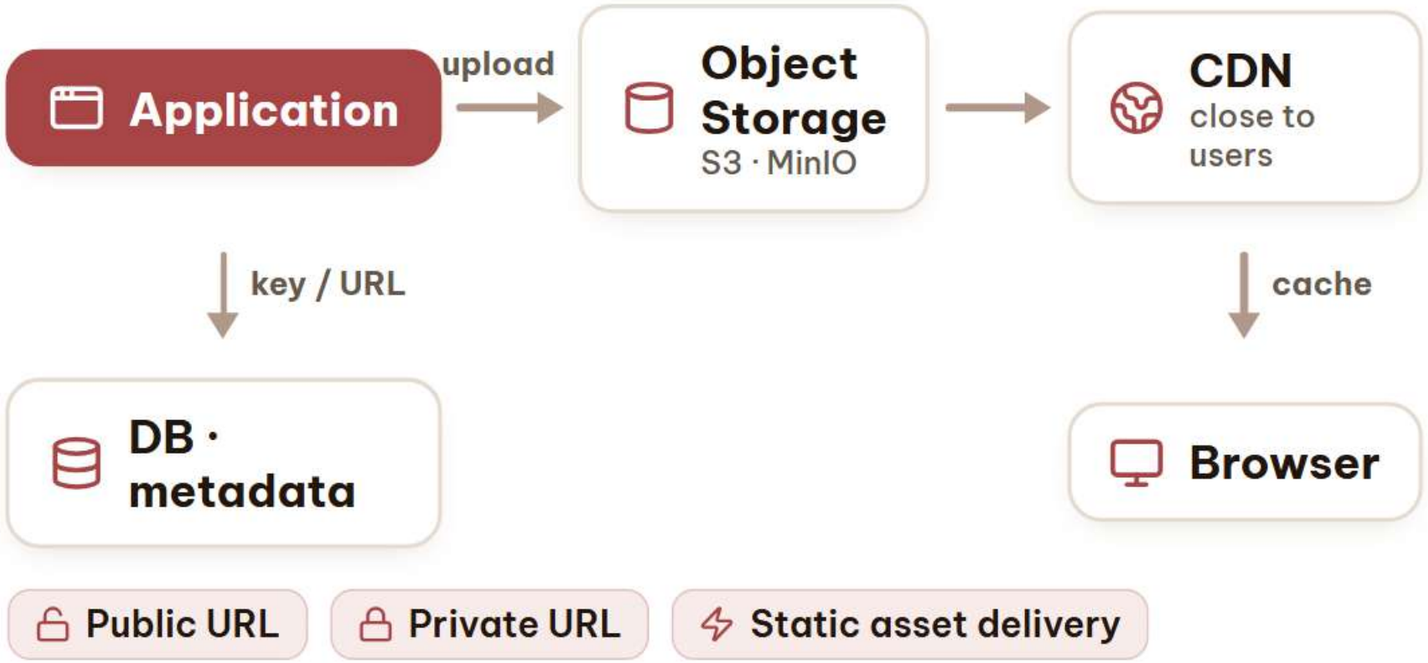
Upload Student Avatar / Course Thumbnail

- Select an image
- Validate type
- Validate size
- Upload
- Save metadata
- Image persists after reload

DELIVERABLE
Working upload feature

As files grow, where should you store & serve them?

The app server runs logic and stores & serves every file → overloaded



SHARED EXERCISE · 40′

Move uploads to object storage

- MinIO / S3-compatible
- Upload
- Save key / URL
- Render images from storage

DELIVERABLE

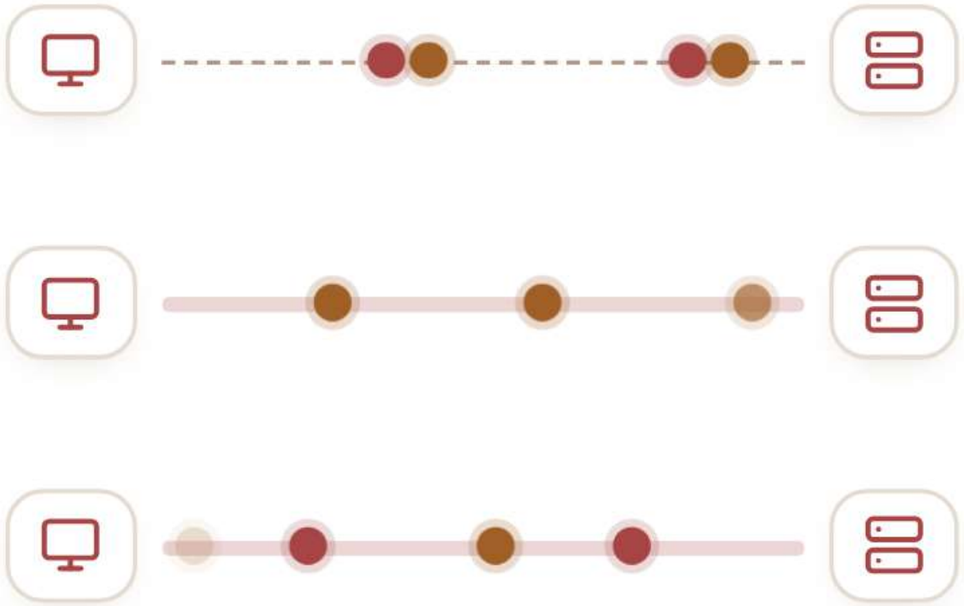
Working object-storage uploads

Can the browser receive new data instantly, without a reload?

Polling
Browser asks repeatedly

SSE
One-way server push

WebSocket
Two-way connection



● browser → server ● server → browser ▬ persistent connection

SHARED EXERCISE · 40'

Realtime notifications or mini chat

Tab A · take action → Tab B · receive instantly

Open 2 browsers / tabs

No reload in Tab B

DELIVERABLE
One working realtime feature

A good library already solves a hard feature — how do you integrate it safely?



SHARED EXERCISE · 40'

Rich-text editor for Course Description

- Edit & format
- Save to DB
- Reload
- Render correctly again
- Handle content safely

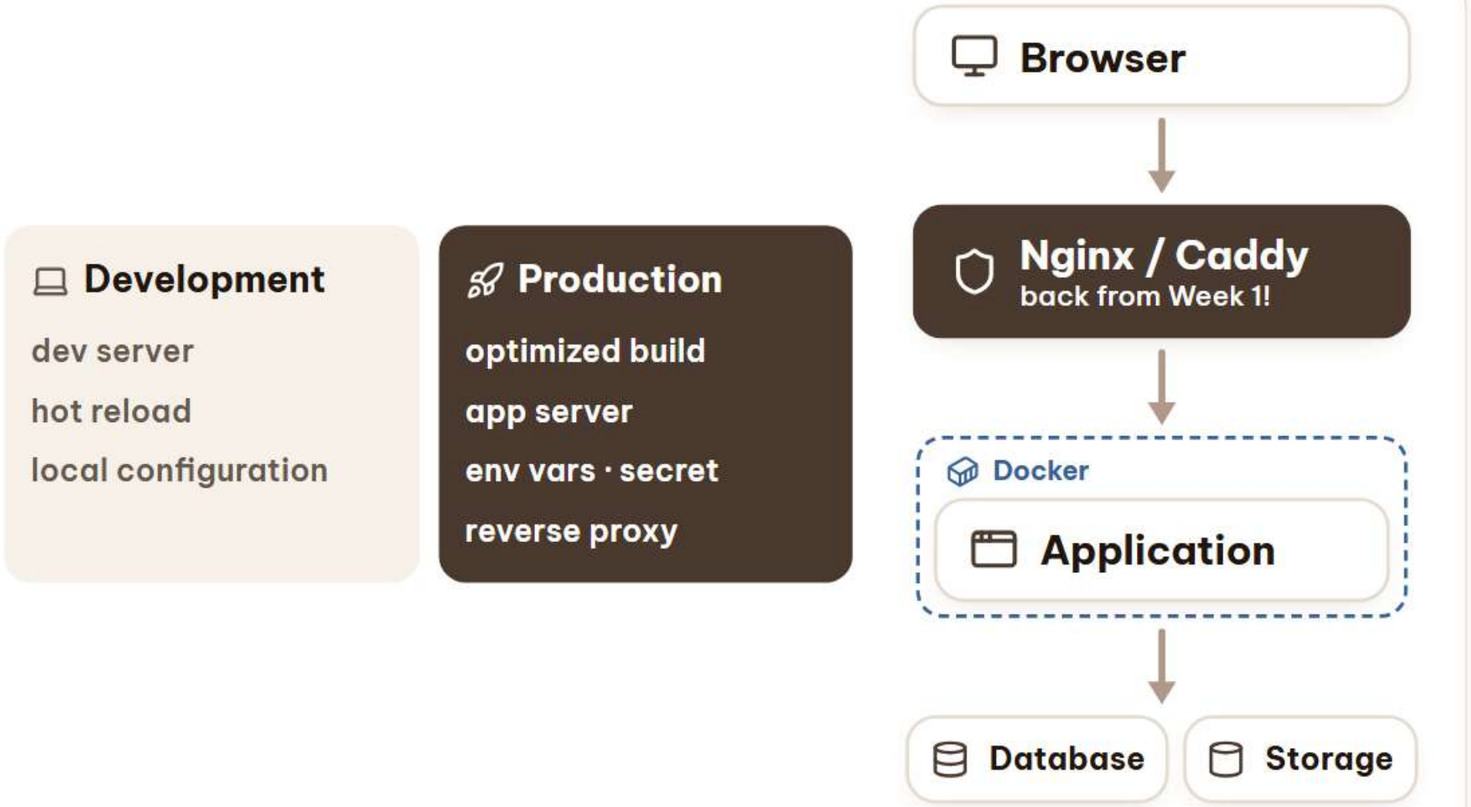
DELIVERABLE

Working rich-content feature

A QUESTION TO CONSIDER

When should you use a third-party solution instead of building from scratch?

“It works on my machine” — is it ready to deploy?



KEYWORDS build · runtime · env vars · secret · static hosting · reverse proxy

SHARED EXERCISE · 40’

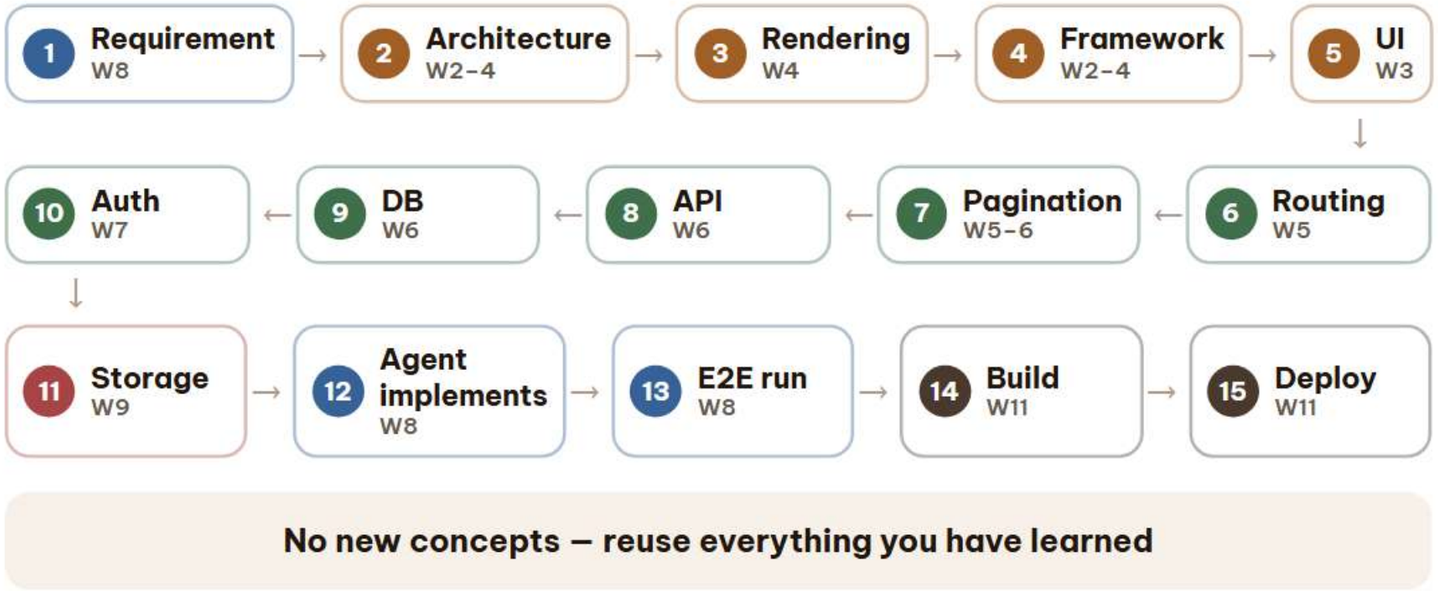
Build & run the production version

- Production build
- Environment variables
- App server
- Reverse proxy or Docker
- No dev server

DELIVERABLE

Working production-style deployment

From requirements to a working product — put it all together



No new concepts — reuse everything you have learned

CHALLENGE · 40′

Take a new feature through the full cycle

Example: **Course Announcement** — rich text, images, visibility, paginated list by course

- Write acceptance criteria
- Delegate to the agent
- Agent runs the user flow
- Fix bugs
- Build / deploy
- Demo

DELIVERABLE

Complete feature in a production-like environment

OVERVIEW

Map of 22 blocks

Click a block for details

Week 1	Week 2	Week 3	Week 4
Agentic Web Dev + HTML/CSS/JS → Static profile site	**Server-rendered Web** → Student List SSR	**Component-based UI** → Student Management components	**CSR · SSR · SSG · Hydration** → Course Catalog × 2
Browser ↔ Web Server → Host Nginx/Caddy	**Client-rendered Web** → Student List CSR	**Modern styling** → 2 styling approaches	**Full-stack frameworks** → Hybrid mini app

Week 5	Week 6	Week 7	Week 8
Routing + query params → Multi-page app	**API + pagination/search/sort** → Paginated Student API	**Authentication / Authorization** → Login + roles	**Requirement → task** → Feature spec
Data fetching + pagination → Paginated Course List	**DB/ORM + paginated query** → Pagination at the DB	**Full-stack flow** → Enrollment feature	**Agentic implementation** → Agent implement + E2E

Week 9	Week 10	Week 11	
File upload → Avatar / thumbnail upload	**Realtime Web** → Notification / chat	**Build · runtime · deploy** → Production deployment	Web Platform Rendering & UI Structure & Data Agentic Dev Practical capabilities Ship
Object storage / CDN → Move to object storage	**Third-party integration** → WYSIWYG Course Description	**Putting it all together** → Full Web Dev Challenge	

LEARNING OUTCOMES

By the end of the course, you can

- **Understand the system you build**
- **Describe requirements clearly**
- **Choose rendering & architecture**
- **Assign tasks to a coding agent**
- **Run & verify the result**
- **Explain how the feature works**
- **Take responsibility for the product**

Coding agents are not a standalone chapter — they are the default way of working throughout the course.